

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Spring Boot Database Handling

Table of Contents

Learning Outcome:	3
1. Database handling in Spring Boot applications	4
Importance and benefits of using Spring Boot for database operations:	5
2. Spring Boot JPA CRUD Application Development	6
Spring Data JPA	6
Setting up JPA dependencies and configuration	7
Creating entity classes and defining relationships	7
Implementing CRUD operations using JPA repositories	8
Handling database transactions	8
3. Using JdbcTemplate for Database Operations	9
JdbcTemplate in Spring Boot	9
Configuring JdbcTemplate in a Spring Boot application	10
Performing basic database operations using JdbcTemplate	10
Executing SQL queries and retrieving data using JdbcTemplate	11
Handling transactions with JdbcTemplate	11
4. Handling Multiple Data Sources in Spring Boot	12
Handling Multiple Data Sources	12
Need for multiple data sources	13
Configuring multiple data sources in a Spring Boot application	13
Creating separate data source configurations for different databases	14
Implementing CRUD operations with multiple data sources	14
Managing transactions across multiple data sources	15
5. CRUD Operations in Spring Boot	16
Introduction to CRUD Operations	16
Implementing create operations using JPA and JdbcTemplate	16
Retrieving data with various querying techniques	17
Updating and deleting records in the database	17
Error handling and validation in CRUD operations	18

UNIT 5: Spring Boot Database Handling

Objectives

In this Unit you will learn –

- Spring Boot Database Handling
- Spring Boot JPA CRUD Application Development
- Using JdbcTemplate for Database Operations
- CRUD Operations in Spring Boot

Learning Outcome:

Learn about Spring Boot's Database handling

5. Spring Boot Database Handling

1. Database handling in Spring Boot applications

Database handling is an essential aspect of most modern applications, and Spring Boot provides robust support for working with databases. In Spring Boot, the database handling is facilitated through various technologies such as Spring Data JPA and JdbcTemplate. These technologies simplify the development process and offer numerous benefits, including:

1. **ORM Support:** Spring Boot integrates seamlessly with Object-Relational Mapping (ORM) frameworks like Hibernate. This allows developers to work with databases using object-oriented programming concepts rather than dealing with low-level SQL queries.
2. **Automatic Configuration:** Spring Boot provides auto-configuration capabilities, which means that the database connection and other necessary configurations can be automatically set up based on the application's dependencies and properties.
3. **Simplified Data Access:** Spring Boot offers high-level abstractions for data access, such as Spring Data JPA and JdbcTemplate. These abstractions provide simplified APIs to perform common database operations like querying, updating, and deleting records.
4. **Database Independence:** Spring Boot promotes database independence, allowing developers to switch between different database vendors without changing the application code significantly. This is achieved through the use of standard SQL and ORM frameworks.
5. **Transaction Management:** Spring Boot provides transaction management capabilities, ensuring that database operations are performed in a consistent and

reliable manner. Transactions can be managed declaratively using annotations or programmatically using the Spring Transaction API.

6. Enhanced Productivity: By leveraging Spring Boot's database handling features, developers can focus more on application logic rather than dealing with low-level database interactions. This leads to increased productivity and faster development cycles.

Importance and benefits of using Spring Boot for database operations:

1. Rapid Application Development: Spring Boot simplifies the process of working with databases by providing a range of abstractions and pre-built components. This allows developers to quickly develop database-driven applications with minimal configuration and boilerplate code.

2. Increased Maintainability: Spring Boot promotes clean and modular code practices, making it easier to maintain and update database-related code. The use of abstractions like Spring Data JPA helps reduce code duplication and promotes code reuse.

3. Seamless Integration with Spring Ecosystem: Spring Boot seamlessly integrates with other Spring projects and modules, such as Spring Security and Spring MVC. This integration allows for a cohesive and unified development experience when building database-driven applications.

4. Testing and Mocking Support: Spring Boot provides robust support for testing database-related code. It offers utilities and annotations that simplify writing unit tests for database operations, including the ability to mock database interactions for isolated testing.

5. **Production-Ready Features:** Spring Boot includes various production-ready features that enhance database handling, such as connection pooling, database health checks, and monitoring. These features help ensure the stability and performance of database operations in production environments.

6. **Community Support and Resources:** Spring Boot has a vast and active community of developers, which translates into excellent support, resources, and documentation. The community-driven nature of Spring Boot ensures that developers can find assistance and guidance when working with databases.

Spring Boot provides a powerful and developer-friendly framework for handling databases in applications. By leveraging Spring Boot's features, developers can achieve efficient and maintainable database operations, leading to faster development cycles and high-quality applications.

2. Spring Boot JPA CRUD Application Development

Spring Data JPA

Spring Data JPA is a powerful feature of the Spring Framework that simplifies database operations by providing a high-level abstraction over Java Persistence API (JPA). It combines the benefits of JPA, such as object-relational mapping and automatic query generation, with the productivity and ease-of-use of the Spring Framework. Spring Data JPA eliminates the need for writing boilerplate code for common database operations, making it an ideal choice for building CRUD (Create, Read, Update, Delete) applications.

Setting up JPA dependencies and configuration

To get started with Spring Data JPA, you need to set up the necessary dependencies and configuration in your Spring Boot application. Follow these steps:

1. Add the Spring Data JPA dependency to your project's build file (e.g., Maven or Gradle).
2. Configure the database connection properties in the application.properties (or application.yml) file, including the database URL, username, password, and driver class.
3. Enable JPA repositories by using the `@EnableJpaRepositories` annotation in your application's main class or a configuration class. This annotation enables Spring Data JPA to scan for repository interfaces and automatically generate implementations.

Creating entity classes and defining relationships

Entity classes in Spring Data JPA represent the database tables or collections. These classes are annotated with JPA annotations to define their properties and relationships. Follow these guidelines when creating entity classes:

1. Annotate the class with the `@Entity` annotation to mark it as a persistent entity.
2. Use the `@Table` annotation to specify the name of the corresponding database table.
3. Define properties as private variables and annotate them with appropriate JPA annotations such as `@Id`, `@Column`, and `@ManyToOne`, to map them to the corresponding database columns and relationships.

4. Establish relationships between entities using annotations such as `@OneToOne`, `@OneToMany`, `@ManyToOne`, or `@ManyToMany`.

Implementing CRUD operations using JPA repositories

Spring Data JPA provides powerful features to perform CRUD operations without writing explicit SQL queries. To implement CRUD operations, follow these steps:

1. Create an interface that extends the `JpaRepository` interface provided by Spring Data JPA. This interface will serve as the repository for a specific entity.
2. Add additional query methods to the repository interface by following Spring Data JPA's naming conventions. For example, to retrieve entities by a specific property, create a method with a naming pattern like `findBy<PropertyName>`.
3. Optionally, you can define custom query methods using the `@Query` annotation and JPQL (Java Persistence Query Language) or native SQL queries.
4. Inject the repository interface into your service or controller classes and use its methods to perform CRUD operations. Spring Data JPA will automatically generate the necessary SQL queries based on the method names or custom queries.

Handling database transactions

Database transactions are crucial to ensure data integrity and consistency. Spring Data JPA simplifies transaction management through declarative transaction support. Follow these steps to handle transactions in your Spring Boot application:

1. Annotate the service or repository methods that require transaction management with the `@Transactional` annotation. This annotation ensures that the method runs within a transactional context.

2. By default, `@Transactional` uses the default transaction manager configured in your application. However, you can specify a specific transaction manager if needed.
3. Transactions can be configured to support different isolation levels and propagation behaviors using the attributes of the `@Transactional` annotation.
4. In case of multiple database operations that need to be treated as a single unit of work, annotate the method with `@Transactional` and handle exceptions appropriately to ensure transaction rollback when necessary.

Spring Boot JPA CRUD application development using Spring Data JPA simplifies database operations by providing a high-level abstraction and eliminating the need for writing repetitive code. By following the steps mentioned above, you can set up JPA dependencies, create entity classes, implement CRUD operations, and handle transactions effectively in your Spring Boot applications.

3. Using JdbcTemplate for Database Operations

JdbcTemplate in Spring Boot

JdbcTemplate is a core feature of the Spring Framework that provides a simple and powerful way to perform database operations using JDBC (Java Database Connectivity). It acts as a higher-level abstraction over JDBC, reducing the amount of boilerplate code required for common database operations. JdbcTemplate simplifies the process of executing SQL queries, handling transactions, and working with result sets.

Configuring JdbcTemplate in a Spring Boot application

To use JdbcTemplate in a Spring Boot application, you need to follow these steps for configuration:

1. Add the necessary dependencies to your project's build file (e.g., Maven or Gradle). JdbcTemplate is typically included as part of the Spring Boot Starter JDBC dependency.
2. Configure the database connection properties in the application.properties (or application.yml) file. Specify the database URL, username, password, and driver class.
3. Create a JdbcTemplate bean in your application's configuration class or XML configuration file. This bean will be used to perform database operations.
4. Inject the JdbcTemplate bean into your service or repository classes and use its methods to execute SQL queries and perform database operations.

Performing basic database operations using JdbcTemplate

JdbcTemplate simplifies basic database operations such as inserting, updating, and deleting records. Follow these guidelines to perform these operations using JdbcTemplate:

1. To perform an insert operation, use the `update` method of JdbcTemplate and provide the SQL statement with placeholders for values. You can pass the values as additional arguments to the `update` method.
2. For updating records, use the same `update` method but provide the SQL statement with placeholders for values and pass the values as additional arguments.

3. To delete records, use the ``update`` method and provide the SQL statement with placeholders for values or conditions, if required.

Executing SQL queries and retrieving data using JdbcTemplate

JdbcTemplate provides methods to execute SQL queries and retrieve data from the database. Follow these steps to execute queries and retrieve data using JdbcTemplate:

1. To execute a SQL query that doesn't return any data, use the ``update`` method and provide the SQL statement with placeholders for values, if required.

2. To execute a SQL query that returns a single result, use the ``queryForObject`` method. Provide the SQL statement, placeholders for values, and a ``RowMapper`` implementation to map the result to a Java object.

3. To execute a SQL query that returns multiple results, use the ``query`` method. Provide the SQL statement, placeholders for values, and a ``RowMapper`` implementation to map each row of the result set to a Java object.

4. You can also use the ``query`` method with a ``ResultSetExtractor`` implementation to extract and process the entire result set.

Handling transactions with JdbcTemplate

JdbcTemplate provides support for managing transactions declaratively. Follow these steps to handle transactions using JdbcTemplate:

1. Annotate the service or repository methods that require transaction management with the ``@Transactional`` annotation. This annotation ensures that the method runs within a transactional context.

2. By default, `@Transactional` uses the default transaction manager configured in your application. However, you can specify a specific transaction manager if needed.

3. Transactions can be configured to support different isolation levels and propagation behaviors using the attributes of the `@Transactional` annotation.

4. In case of multiple database operations that need to be treated as a single unit of work, annotate the method with `@Transactional` and handle exceptions appropriately to ensure transaction rollback when necessary.

JdbcTemplate simplifies database operations in a Spring Boot application by providing a convenient and efficient way to execute SQL queries, perform CRUD operations, and handle transactions. By following the guidelines mentioned above, you can configure

JdbcTemplate, perform basic database operations, execute SQL queries, retrieve data, and handle transactions effectively in your Spring Boot applications.

4. Handling Multiple Data Sources in Spring Boot

Handling Multiple Data Sources

In some scenarios, a Spring Boot application may need to connect to multiple databases. This could be due to different data sources for different modules or the need to integrate with legacy systems. Handling multiple data sources allows you to effectively manage and interact with multiple databases within a single Spring Boot application.

Need for multiple data sources

The need for multiple data sources can arise in various situations, including:

1. **Microservices Architecture:** When implementing microservices, each service may have its own database, requiring multiple data sources.
2. **Legacy Systems:** Integrating with legacy systems often involves connecting to multiple databases to access different data sources.
3. **Sharding and Partitioning:** Distributed databases may require sharding or partitioning, where each shard or partition represents a separate data source.
4. **Multi-tenancy:** In multi-tenant applications, each tenant may have its own database, necessitating multiple data sources.

Configuring multiple data sources in a Spring Boot application

To configure multiple data sources in a Spring Boot application, follow these steps:

1. **Define separate configurations for each data source:** Create separate configuration classes for each data source, specifying the necessary connection properties such as URL, username, password, and driver class.
2. **Configure data source beans:** In each configuration class, create a `'DataSource'` bean using the connection properties specific to that data source.
3. **Specify the primary data source:** Use the `'@Primary'` annotation on one of the data source beans to indicate it as the primary data source.

4. Configure JPA EntityManagerFactory and TransactionManager: Configure the 'EntityManagerFactory' and 'TransactionManager' beans for each data source, specifying the corresponding data source bean and other required properties.

Creating separate data source configurations for different databases

To create separate data source configurations for different databases, you can follow these steps:

1. Create a separate configuration class for each data source, annotated with '@Configuration' and '@EnableTransactionManagement'.
2. In each configuration class, define the data source properties, such as URL, username, password, and driver class, specific to that database.
3. Create a 'DataSource' bean in each configuration class using the defined properties.
4. Configure the JPA EntityManagerFactory and TransactionManager beans for each data source, ensuring that the corresponding data source bean is used.

Implementing CRUD operations with multiple data sources

To implement CRUD operations with multiple data sources, you need to:

1. Create separate repositories or DAOs for each data source, specifying the corresponding entity classes and data source configurations.
2. Inject the required repository or DAO beans into your services or controllers to perform CRUD operations on the respective data sources.

3. Use the repository or DAO methods to execute database operations specific to each data source.

Managing transactions across multiple data sources

Managing transactions across multiple data sources requires coordination between the different transaction managers. Follow these steps to manage transactions effectively:

1. Use the `@Transactional` annotation on the service methods that involve database operations across multiple data sources.
2. Configure the transaction manager for each data source by creating separate `PlatformTransactionManager` beans.
3. Specify the `transactionManager` attribute in the `@Transactional` annotation to indicate the corresponding transaction manager to be used.
4. Ensure that each database operation within the transactional method is performed using the appropriate data source or repository.

Handling multiple data sources in a Spring Boot application allows for effective management of multiple databases. By understanding the need for multiple data sources, configuring them correctly, creating separate data source configurations, implementing CRUD operations, and managing transactions across multiple data sources, you can efficiently work with multiple databases within your Spring Boot application.

5. CRUD Operations in Spring Boot

Introduction to CRUD Operations

CRUD (Create, Read, Update, Delete) operations are fundamental operations performed on a database to manage data. In the context of a Spring Boot application, CRUD operations involve creating, reading, updating, and deleting data in the underlying database.

Exploring the CRUD operations:

1. **Create:** The create operation involves inserting new records into the database. It typically requires providing data input and saving it as a new entity in the database.
2. **Read:** The read operation involves retrieving data from the database. It can be performed in various ways, such as retrieving a single record by its identifier or querying the database to fetch multiple records based on certain conditions.
3. **Update:** The update operation involves modifying existing records in the database. It requires retrieving an entity, updating its attributes, and saving the changes back to the database.
4. **Delete:** The delete operation involves removing records from the database. It can be done by specifying the record's identifier or using certain criteria to identify and delete multiple records.

Implementing create operations using JPA and JdbcTemplate

1. **JPA:** To implement create operations using JPA (Java Persistence API), you need to define an entity class with appropriate annotations (`@Entity`, `@Table`, etc.), create a repository interface extending `JpaRepository`, and use the repository's `save()` or `saveAll()` methods to persist the new entity into the database.

2. JdbcTemplate: To implement create operations using JdbcTemplate, you can use the JdbcTemplate class provided by Spring. It requires configuring a DataSource and then using the JdbcTemplate instance to execute an SQL INSERT statement with the necessary parameters.

Retrieving data with various querying techniques

1. JPA: In JPA, you can retrieve data using various querying techniques:

- findById(): Retrieves a single record by its identifier.
- findAll(): Retrieves all records from a specific entity.
- Query methods: Define custom query methods in the repository interface using Spring Data JPA's method naming conventions.
- @Query annotation: Use the @Query annotation to define custom SQL or JPQL queries.

2. JdbcTemplate: With JdbcTemplate, you can retrieve data by executing SQL SELECT statements using methods like query(), queryForObject(), or queryForList(). These methods allow you to pass SQL queries and retrieve data as objects or collections.

Updating and deleting records in the database

1. JPA: To update records with JPA, retrieve the entity to be updated using a find operation, modify its attributes, and save the changes using the repository's save() method.

2. JdbcTemplate: For updating records with JdbcTemplate, execute an SQL UPDATE statement with the necessary parameters using the JdbcTemplate instance.

For deleting records, use the repository's `delete()` or `deleteById()` methods with JPA. With `JdbcTemplate`, execute an SQL `DELETE` statement with the necessary parameters using the `JdbcTemplate` instance.

Error handling and validation in CRUD operations

1. Error handling: When performing CRUD operations, it is essential to handle potential errors that may occur. This includes handling exceptions such as `DataAccessException`, `ConstraintViolationException`, or any custom exceptions that might be thrown during database operations.

2. Validation: Validating input data before performing CRUD operations helps ensure data integrity. Spring provides validation support through annotations such as `@NotNull`, `@Size`, and custom validators. Use these annotations to validate input data before saving or updating entities.

CRUD operations form the foundation of working with databases in Spring Boot applications. By understanding how to implement create operations using JPA and `JdbcTemplate`, retrieve data using querying techniques, update and delete records, and handle errors and validation, you can effectively perform CRUD operations in your Spring Boot applications, ensuring efficient data management.